

From Theory to Practice: MLP with PyTorch

1. Training Pipeline

Based on what we built in this part, we currently have the following:

- **Architecture:** 784 (input layer) $\rightarrow$ Linear(784,256) + ReLU $\rightarrow$ Linear(256,128) + ReLU $\rightarrow$ Linear(128,10) (corresponding to our classes from our dataset). Total : 235 146 parameters (200 960 + 32 896 + 1 290), spread across 3 Linear layers.
- **Loss function:** nn.CrossEntropyLoss, which fits our classification usecase (10 clothing categories) whereas nn.MSELoss is reserved for continuous value prediction and not classification.
- **Training configuration:** Adam Optimizer, lr= 10^{-3} , weight_decay= 10^{-4} , batch_size=64, 15 epochs, no BatchNorm, no Dropout.
- **Baseline results:** The train-validation gap is initially negative (epoch 1: train 82.33% / validation 85.77%, gap -3.44 pts), then becomes positive and gradually increases (epoch 5: +2.25 pts; epoch 10: +3.41 pts; epoch 15: +4.36 pts). The training loss decreases monotonically (0.4838 $\rightarrow$ 0.1787), whereas the validation loss reaches a minimum around epoch 7 (0.3150) and then fluctuates/increases (0.3429 at epoch 15) without continuing to decrease. This widening gap, combined with the stagnation of the validation loss while the training loss continues to decrease, is a characteristic sign of the onset of overfitting in the baseline model on Fashion-MNIST.

2. Experimental Analysis

2.1. Experiment 1: Architecture

As the graphs show us in this experiment:

- **Underfitting :** Micro [4] has the smallest gap (0.58 pt) but also the lowest accuracy (train 82.29% / val 81.71%): a small gap therefore does not mean good generalisation when both train and val are low — here the model lacks the capacity to learn the task (underfitting), and train and val plateau together.
- **Overfitting :** Small [64], Baseline and Large show a markedly larger gap (3.31 / 5.50 / 5.47 pt) with a high train acc (92–94%) while val acc stays around 88.5–89%: the extra capacity is being used to memorise the training data rather than to generalise.
- **Val accuracy plateau:** val acc plateaus as early as Small [64] (88.81%) and barely progresses further with Baseline (88.52%) or Large (88.97%, +0.16 pt vs Small): beyond 64 hidden neurons, adding capacity brings almost no generalisation benefit under this protocol. Bias-variance trade-off: the final gap is not strictly monotonic with capacity (0.58% $\rightarrow$ 3.31% $\rightarrow$ 5.50% $\rightarrow$ 5.47%): it rises sharply then saturates/decreases very slightly between Baseline and Large.
- **Architecture choice:** Small [64] offers the best trade-off (val acc nearly identical to the larger configurations, 88.81%, for a much smaller gap, 3.31 pt, and far fewer parameters); Baseline [256,128] nevertheless remains a reasonable choice going forward, since its more pronounced overfitting can be addressed specifically by the regularisation studied in Experiment 3

2.2. Experiment 2: Optimizer and Learning Rate

2.2.1. Sub-Experiment 2.A: Learning Rate

With $\text{lr}=10^{-5}$, accuracy stays very low (33.66% after 15 epochs): the update steps are too small to meaningfully move the weights within the allotted epoch budget, so the model is essentially undertrained and close to random behaviour. With $\text{lr}=10^{-3}$, learning progresses clearly (81.81% val) but remains far from the optimum reached with $\text{lr}=0.1$ (89.26% val).

Given the curves, the speed at which they converge is clearly different depending on the learning rate. With $\text{lr}=0.1$, the train loss drops sharply within the first few epochs and the curve flattens out at around epoch 12, at which point we can consider that the train loss has reached an acceptably low value. With $\text{lr}=10^{-3}$, by contrast, the train loss decreases much more slowly and has clearly not plateaued by epoch 14.

Between $\text{lr}=10^{-3}$ and $\text{lr}=0.1$, the val accuracy gap is 7.45 pt for the same number of epochs: a suboptimal LR choice therefore has a direct and substantial cost on SGD's convergence speed, illustrating its strong sensitivity to the learning rate.

2.2.2. Sub-Experiment 2.B: Optimizer Comparison

Each optimiser reaches its best val accuracy at a different LR: SGD at $\text{lr}=0.1$ (89.70%), Adam at $\text{lr}=10^{-3}$ (89.01%). At their respective best LR, Adam reaches its best val accuracy as early as epoch 9, versus 15 epochs for SGD (which keeps improving until the end): Adam therefore converges noticeably faster, which is explained by its per-parameter adaptive updates, which automatically adjust the effective step size unlike SGD's fixed global LR. At the same $\text{lr}=10^{-3}$, SGD only reaches 81.81% (Exp. 2A) while Adam reaches 89.01%: the same nominal LR value therefore does not transfer from one optimiser to another, since Adam's effective update scale fundamentally differs from that of SGD.

2.3. Experiment 3: Regularisation

Dropout and weight decay reduce overfitting through different mechanisms:

- Dropout randomly disables a fraction of activations at each step, which prevents neuron co-adaptation and amounts to averaging over an ensemble of sub-networks;
- Weight decay adds an L2 penalty on the weight norm to the loss function, which constrains the weights to stay small and smooths the decision boundary.

The smallest gap is obtained by Dropout + BN (-1.34 pt, val > train), but with the lowest train acc (86.84%, a sign of slight underfitting from the combined regularisation). The highest val accuracy is technically No regularisation (89.07%), but only marginally ahead of Dropout alone (89.04%, a 0.03 pt gap), while the latter reduces the gap from 6.24 pt to 1.70 pt and achieves the best test acc of all configurations (88.85%): Dropout alone therefore offers the best practical trade-off between generalisation and stability. Regarding the epoch of best val acc, adding regularisation generally delays the point at which the model reaches its validation optimum: No regularisation peaks early (epoch 16/20), Weight decay even earlier (epoch 15), while Dropout alone peaks latest (epoch 19) and Dropout + BN at epoch 17. Dropout injects noise into the gradient signal at every epoch, which slows convergence toward the validation optimum. This tells us that slower training can reduce train-val gap.